Texture Mapping (1)

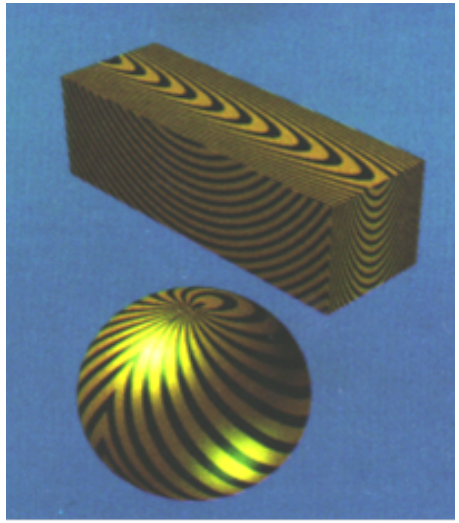
Contents

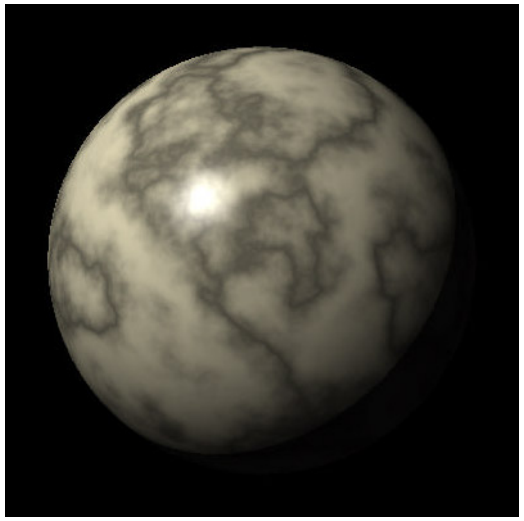
- Introduction
- 2D texture mapping
- Surface parameterization
- Polygon mapping
- Cylindrical and spherical mapping
- Two phase mapping
  - S-mapping
  - O-mapping
  - Issues
  - Examples
- Mapping through modeling
- Mapping onto bicubic parametric patches
- Mipmapping

Introduction

- Texture – graphics pattern based on matrix of pixel
- Texture mapping – transposition of graphics pattern onto the surface of an object
- Through texture are presented more attributes of the object surface, rather than simply the color by the Phong reflection model
- There are three major considerations in the texture mapping:
  1. What attribute or parameter of the model or object is to be modulated to produce the desired textural effect?
  2. Texture mapping method. Define the function of correspondence between the texture domain and the world space of the object
  3. Anti-aliasing processing

Introduction





Attributes and parameters

- In 1986 Heckbert classified the parameters that should be modulated to provide the textural impression:

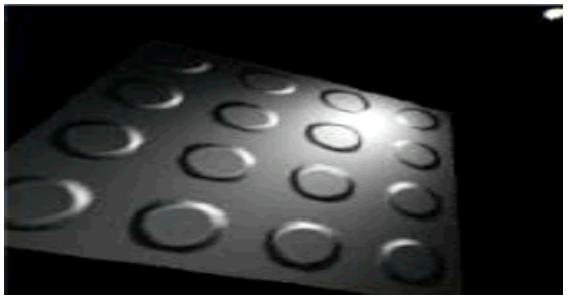
  (Heckbert P.S. "Survey of texture mapping", IEEE Computer Graphics and Applications, 6(11), pp. 56-67)

  1. *Surface color* (diffuse reflection coefficients) – the most commonly used parameter for texture mapping. Propose by Catmull in 1974
  2. *Specular and diffuse reflection* (environment mapping) – known as environment mapping. Developed by Blinn and Newell in 1976
  3. *Normal vector perturbation* – it is a modification of the reflection model, proposed by Blinn in 1978, that gives the appearance of a perturbed surface
  4. *Specularity* – modulate a function of the surface roughness. The approach was proposed by Blinn in 1978
  5. *Transparency* – proposed by Gardener in 1985. It is in fact a method of generating a complete object (e.g. a cloud, chemically etched glass)

- All texture mapping methods, excepting the environment mapping, bind the texture onto the object, and the texture then becomes a permanent part of the object, invariable under object transformations and viewing conditions

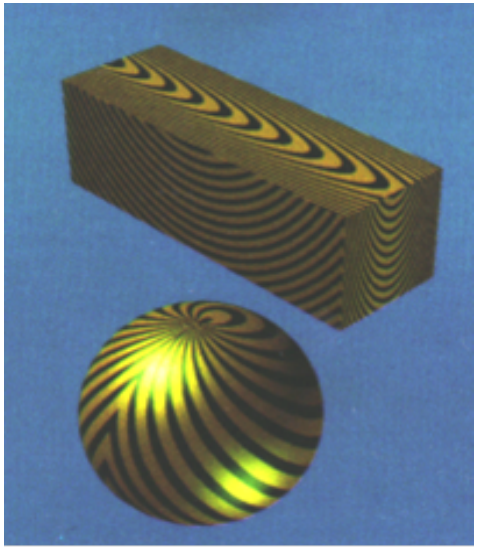
Mapping approach

1. *Bound texture* – a 2D texture pattern is mapped onto the surface of the object. Then the texture remains attached to the surface
2. *View dependent mapping* – the texture pattern watched by the viewer depends on the viewing direction. For instance, chrome mapping, environment mapping, reflection and refraction mapping
3. *Bump mapping* – the effect of surface deformation by the modification of the normal vector. Computing the light intensity by the local reflection model produces a textural effect



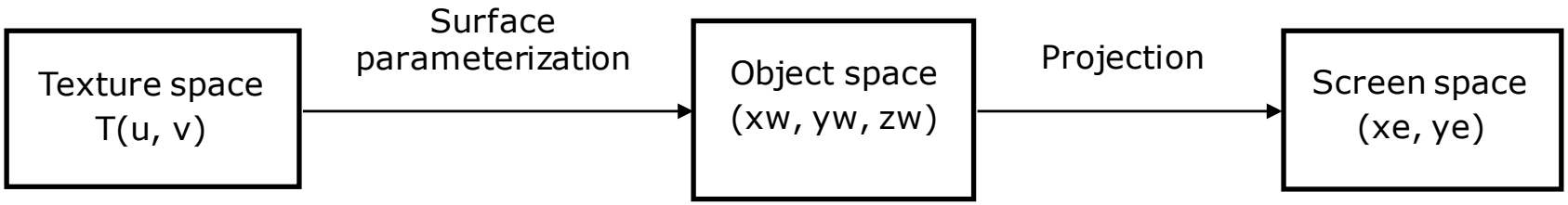
Texture space

1. 2D texture (u,v)
2. 3D texture (u,v,w)



2D texture mapping

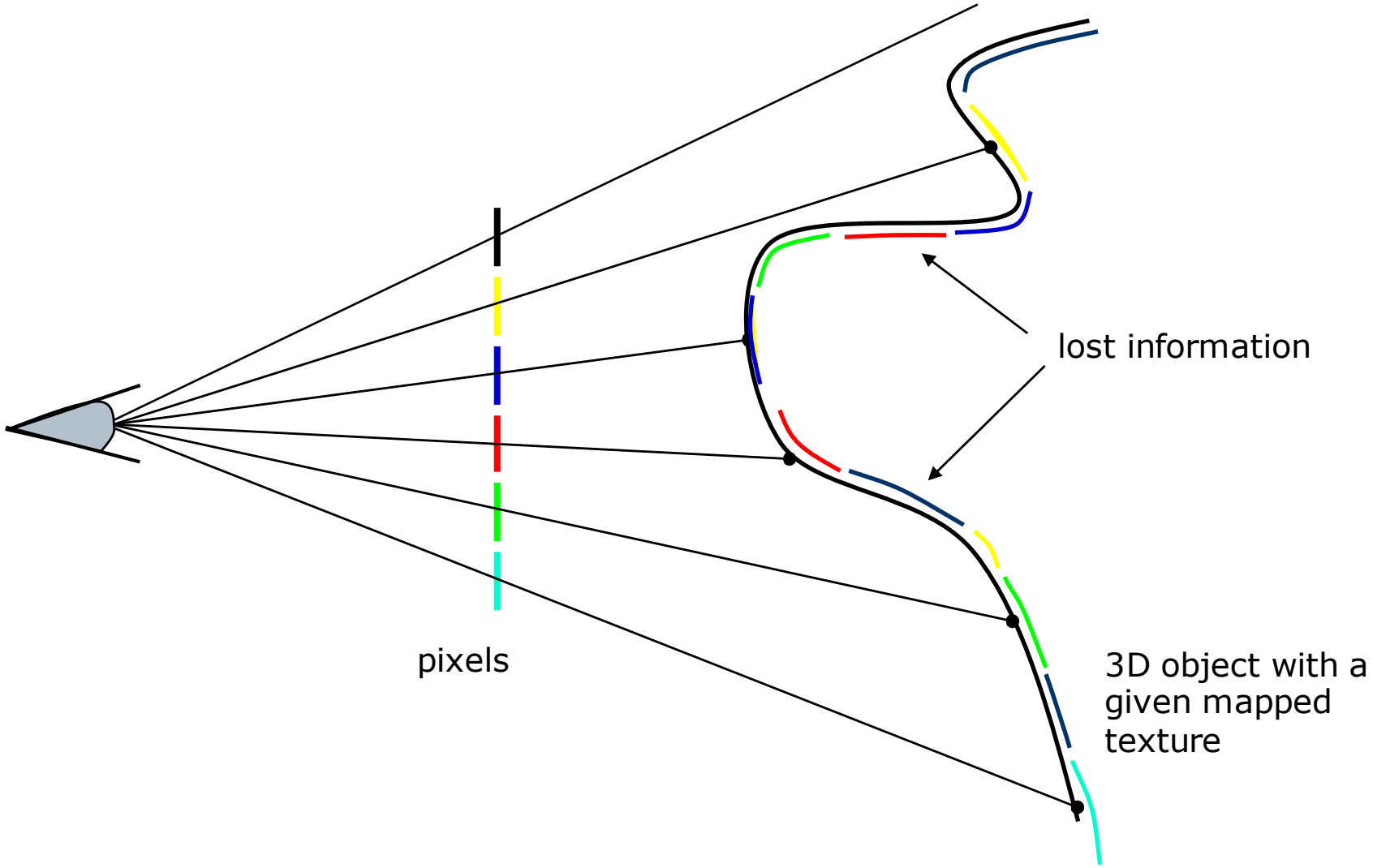
- 2D texture domain



- Main issues:

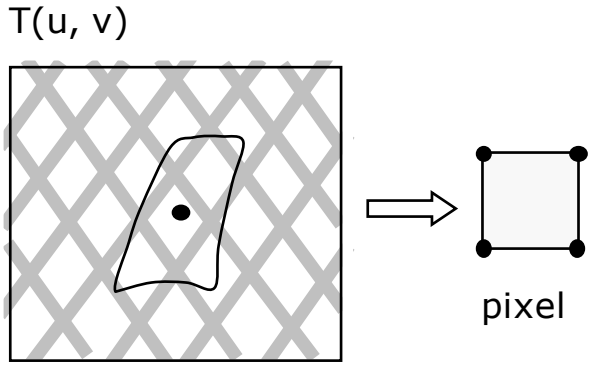
1. Surface parameterization
   - Need to stick a texture pattern onto a polygon mesh object
   - Difficult approximation for discontinuous real objects
   - Simple for cubic parametric surfaces
2. Anti-aliasing
   - Mapping a texture onto adjacent polygons
   - Projection of an area into a pixel -> lost information
   - Keep graphical quality even by lost information projection

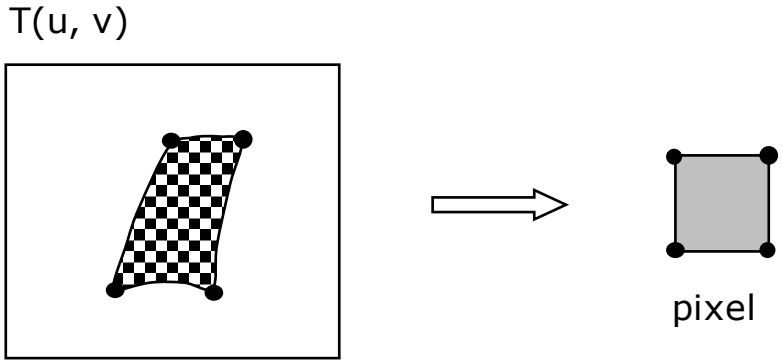
Texture mapping - antialiasing



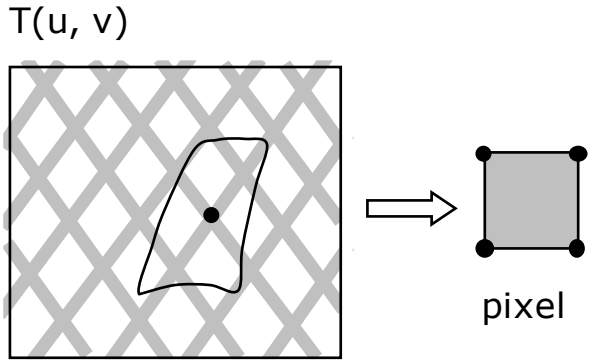
2D texture mapping

- Anti-aliasing
- Computing the pixel value:
  a) Intensity is a function of the filling factor.
  b) Pixel value depends on the mapping of the pixel center.





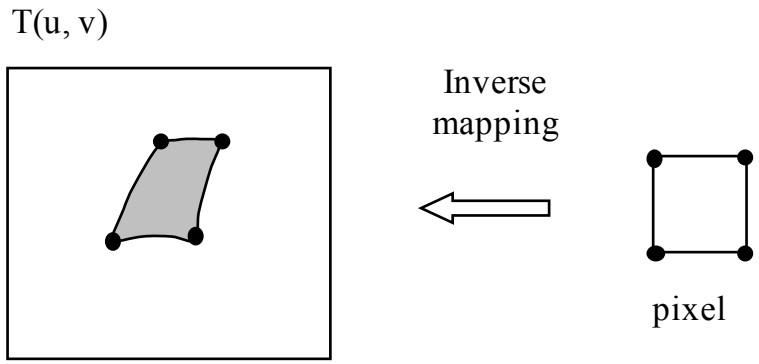
(a)



(b)

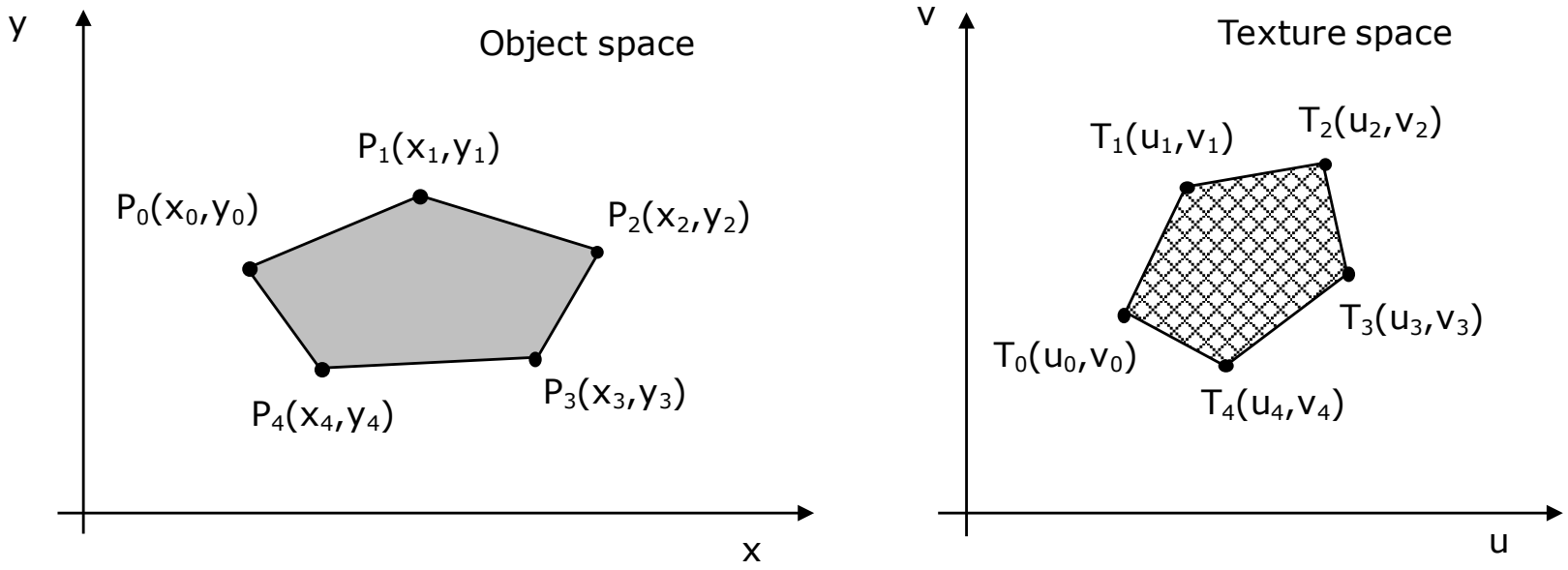
2D texture mapping

- Inverse mapping
  - Find the pre-image of the current pixel in the texture domain
  - Non-linear transformation
  - Pixel maps into an area in texture space that in general is a curvilinear quadrilateral
  - Steps:
    1. Consider four pixel corner points
    2. Invert the object to screen space
    3. Invert the surface parameterization



Surface parameterisation

- Associates a point on the object (x_w, y_w, z_w) with a point (u,v) in texture space
- Polygon mapping
  1. Local problem: mapping each interior point in the polygon into a point in T(u,v) space
  2. Global problem: mapping each polygon, for an entire object, from a number of connected polygons



Surface parameterisation

- Problem:
  - knowing the mapping of the object vertices into the texture space let us find the mapping of each point in the polygon for operations like: Simple rendering, clipping, scale, and deformation
  - Case1: the mapping function for each point is already known – map the clipped polygon as the initial polygon by the rendering of each point
  - Case 2: the mapping function is unknown – should determine the analytical function of correspondence
    1. Linear toward some direction
    2. Non-linear: divide the initial polygon into a set of linear variation areas

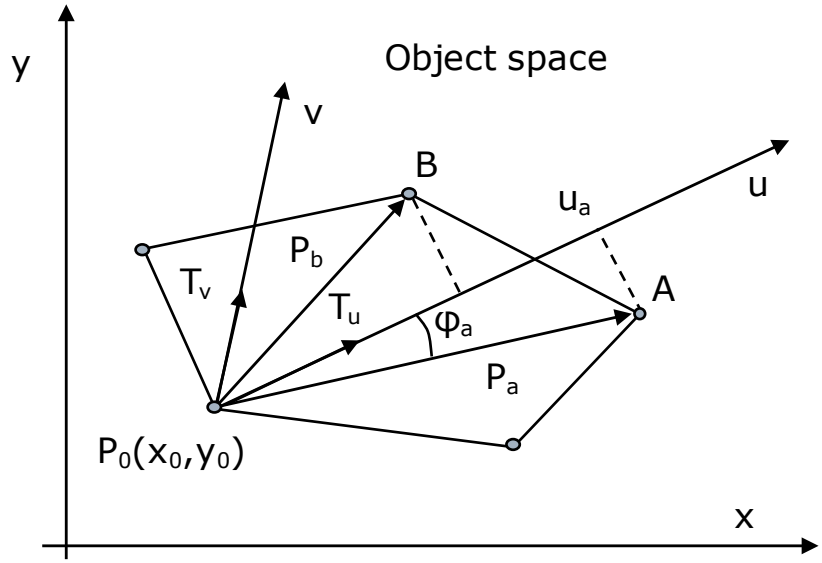
Polygon mapping – linear mapping

Linear variation of u by x and v by y, and (x_0, y_0) maps onto (u_0, v_0):

$$u = T_u \cdot (x - x_0) + u_0$$

$$v = T_v \cdot (y - y_0) + v_0$$

where T_u and T_v are the directions for linear variations of u and v in the object space. Let us compute the T_u and T_v vectors.



Polygon mapping

Considering the vertex A, let us compute the u_a coordinate in texture space as the projection of vector P_a onto u axis. T_u is a normalized vector:

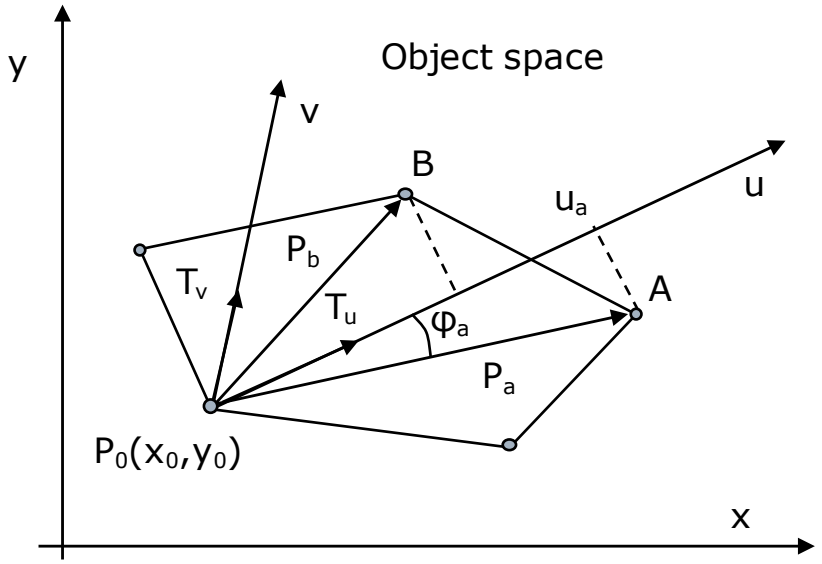
$$u_a = T_u \cdot P_a + u_0 = |T_u| \cdot |P_a| \cos \varphi_a + u_0$$

or $$u_a = t_{ux} (x_a - x_0) + t_{uy} (y_a - y_0) + u_0$$

To compute $T_u(t_{vx}, t_{vx})$ we need to know the texture coordinates at least for two positions A and B in object space. Let be A and B two polygon vertices:

$$u_a = t_{ux} (x_a - x_0) + t_{uy} (y_a - y_0) + u_0$$

$$u_b = t_{ux} (x_b - x_0) + t_{uy} (y_b - y_0) + u_0$$



Polygon mapping

Resolving the two equations for t_{ux} and t_{uy}:

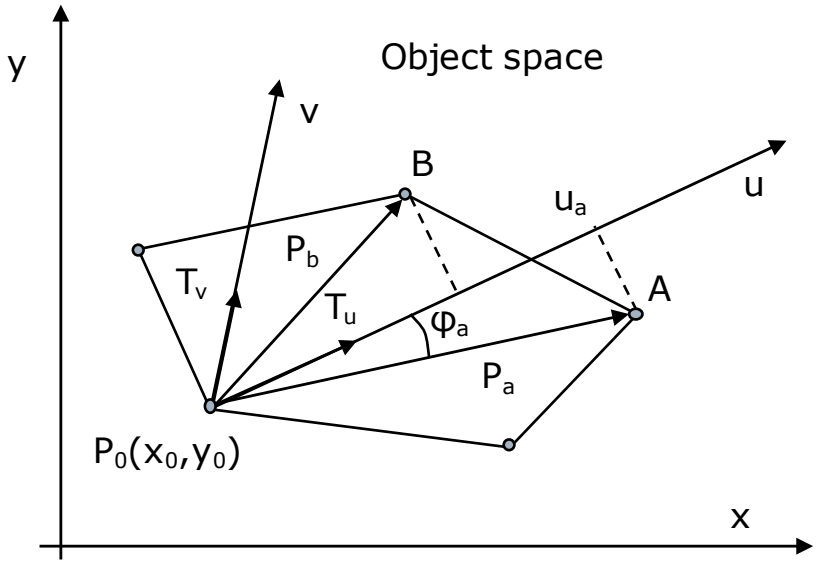
$$t_{ux} = \frac{\frac{u_a - u_0}{y_a - y_0} - \frac{u_b - u_0}{y_b - y_0}}{\frac{x_a - x_0}{y_a - y_0} - \frac{x_b - x_0}{y_b - y_0}} \qquad t_{uy} = \frac{\frac{u_a - u_0}{x_a - x_0} - \frac{u_b - u_0}{x_b - x_0}}{\frac{x_a - x_0}{x_a - x_0} - \frac{x_b - x_0}{x_b - x_0}}$$

Similarly compute $T_v(t_{vx}, t_{vy})$.

Therefore, a point P(x, y) in the object space has the texture coordinates:

$$u = t_{ux}(x - x_0) + t_{uy}(y - y_0) + u_0$$

$$v = t_{vx}(x - x_0) + t_{vy}(y - y_0) + v_0$$



Cylindrical and spherical mappings

- Cylindrical mapping

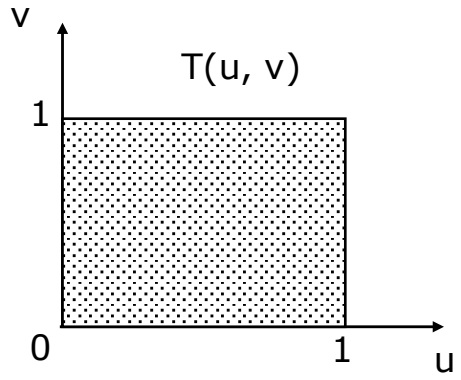
A point P(x, y, z) on the cylinder surface is represented as:

$(r\cos\theta, r\sin\theta, hs)$

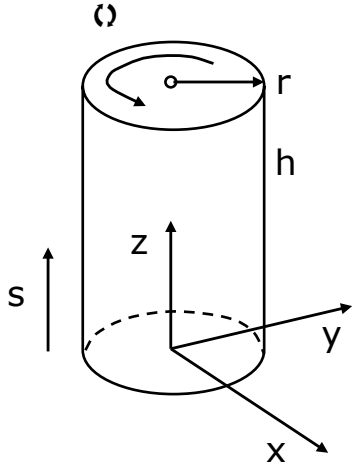
where r radius, h height, $0 < \theta \leq 2\pi$ and $0 < s \leq 1$

The inverse mapping is:

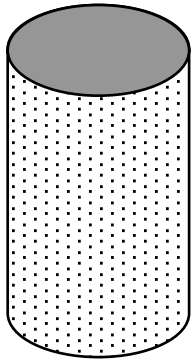
$(u, v) = (\theta/2\pi, s), \quad u, v \in [0, 1]$



(a) Texture domain



(b) Cylinder object



(c) Texture mapped on the cylindrical surface

Cylindrical and spherical mappings

- Spherical mapping

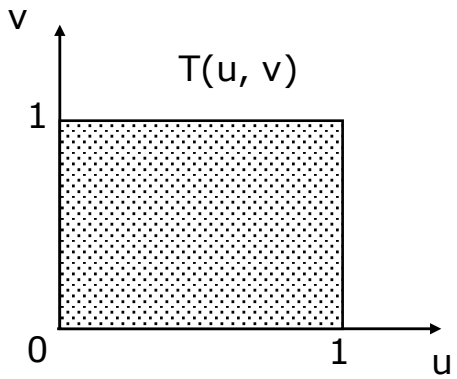
  A point P(x, y, z) on the spherical surface is represented as:

  $(r \cos\theta \sin\varphi,\ r \sin\theta \sin\varphi,\ r \cos\varphi)$

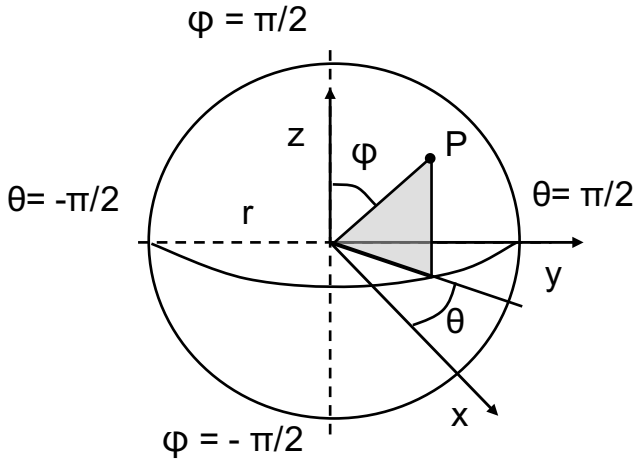
  where r radius, $-\pi < \theta \leq \pi$, $-\pi/2 < \varphi \leq \pi/2$

  The inverse mapping is:

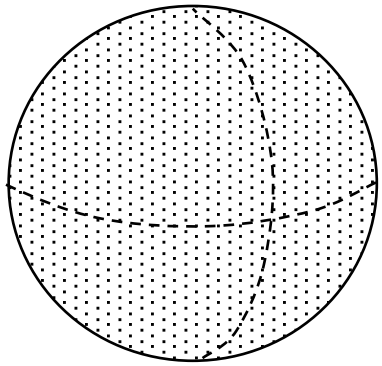
  $(u, v) = ((\theta + \pi)/2\pi,\ (\varphi + \pi/2)/\pi),\quad u, v \in [0, 1]$



(a) Texture domain



(b) Sphere object



(c) Texture mapped on the spherical surface

Cylindrical and spherical mappings

- Proposed exercises:
  1. Compute the inverse mapping of a cylinder surface for
     $\theta_0 < \theta \leq \theta_{max}$ and $s_0 < s \leq s_{max}$
  2. Compute the inverse mapping of a spherical surface for
     $0 < \theta \leq \pi/2$ and $\pi/4 < \varphi \leq \pi/2$
  3. Compute the inverse mapping of a spherical surface for
     $\theta_0 < \theta \leq \theta_{max}$ and $\varphi_0 < \varphi \leq \varphi_{max}$

Two phase mapping

- Proposed in 1986, by E.A. Bier and K.R. Sloan
- Overcomes the global mapping problem
  - mapping each polygon, for an entire object, from a number of connected polygons
- Unifies texture mapping and environment mapping
- Two phases:
  1. S mapping

     Maps the 2D texture space to a simple 3D intermediate surface

     $T(u,v) \rightarrow T'(x_i,y_i,z_i)$
  2. O mapping

     Maps the 3D texture pattern onto the object surface

     $T'(x_i,y_i,z_i) \rightarrow O(x_w,y_w,z_w)$

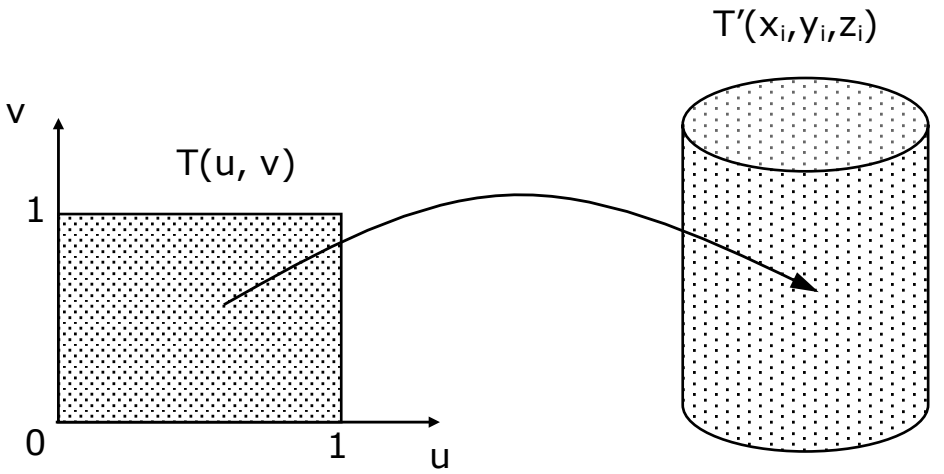
Two phase mapping – S mapping

- S mapping

  Maps the 2D texture space to a simple 3D intermediate surface

  $T(u,v) \rightarrow T'(x_i,y_i,z_i)$

- Intermediate surfaces (Bier):
  1. Plane at any orientation
  2. Curved surface of a cylinder
  3. Faces of a cube
  4. Surface of a sphere



(a) Texture domain

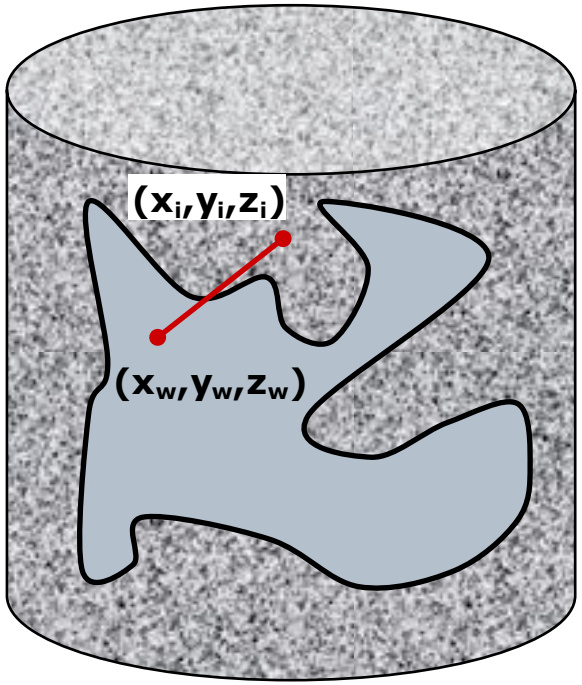
(b) Texture mapped on the cylindrical surface

Two phase mapping – O mapping

- O mapping

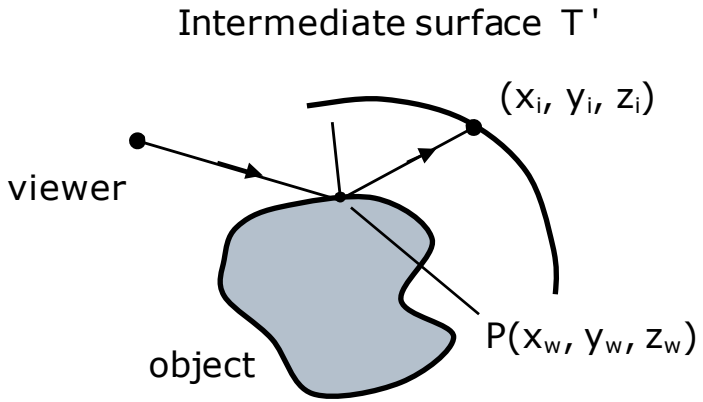
  Maps the 3D texture pattern onto the object surface

  $T'(x_i,y_i,z_i) \rightarrow O(x_w,y_w,z_w)$

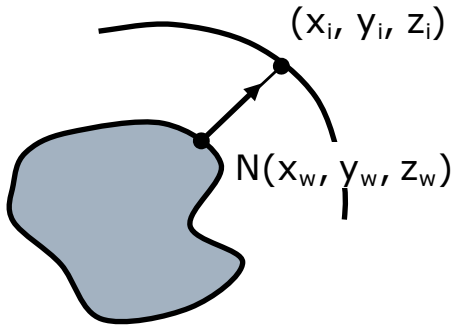


$T'(x_i,y_i,z_i)$

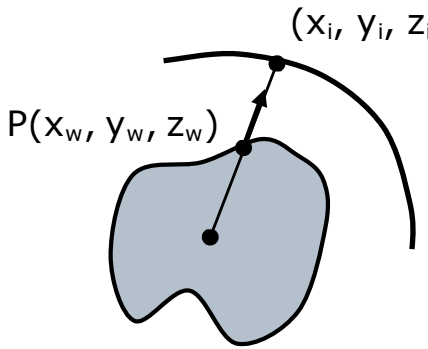
Two phase mapping – O mapping



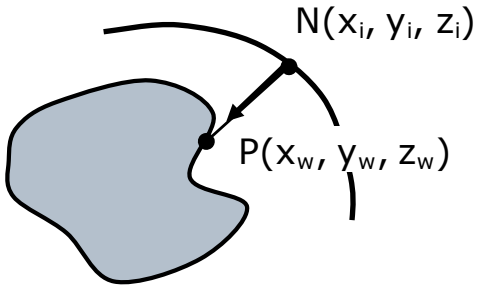
(a) Intersection of the reflected view ray with the intermediate surface T' (environment mapping)



(b) Intersection of the surface normal with T'



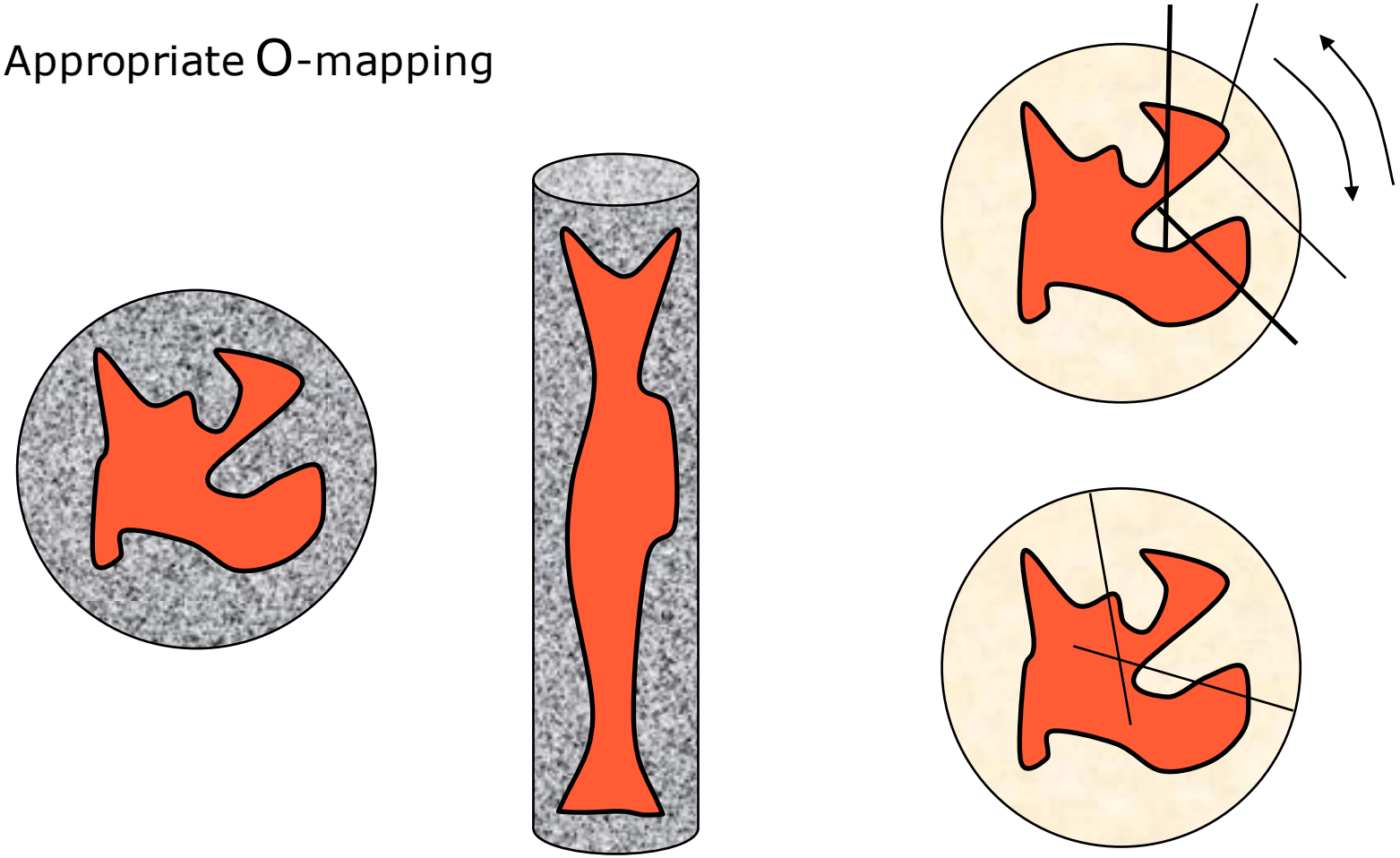
(c) Intersection of the line through object point and the object centroid with T'



(d) Intersection of the line from object point to T' whose orientation is given by the intermediate surface normal

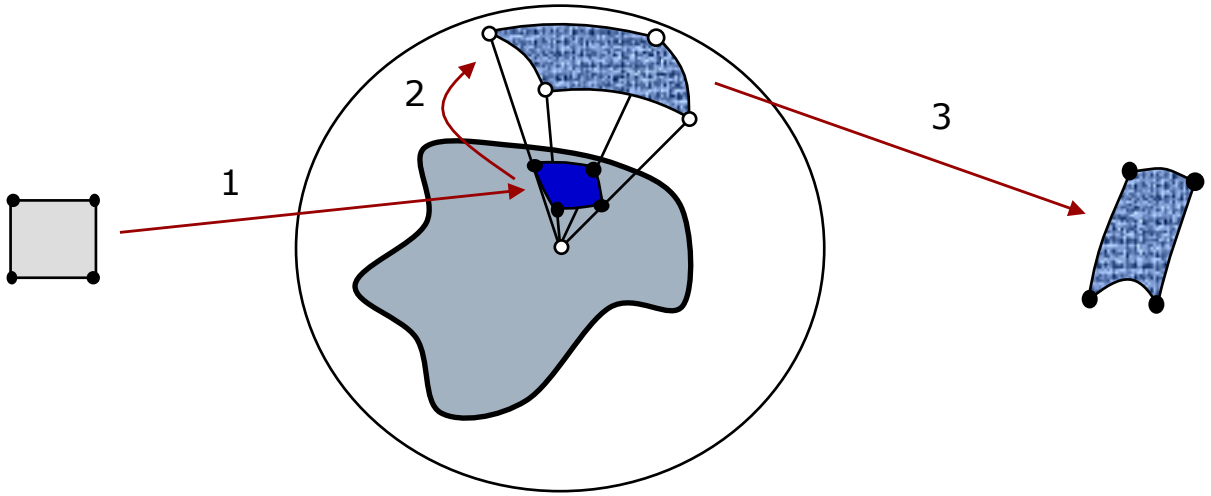
Two phase mapping – issues

- Appropriate intermediate surface
- Appropriate O-mapping

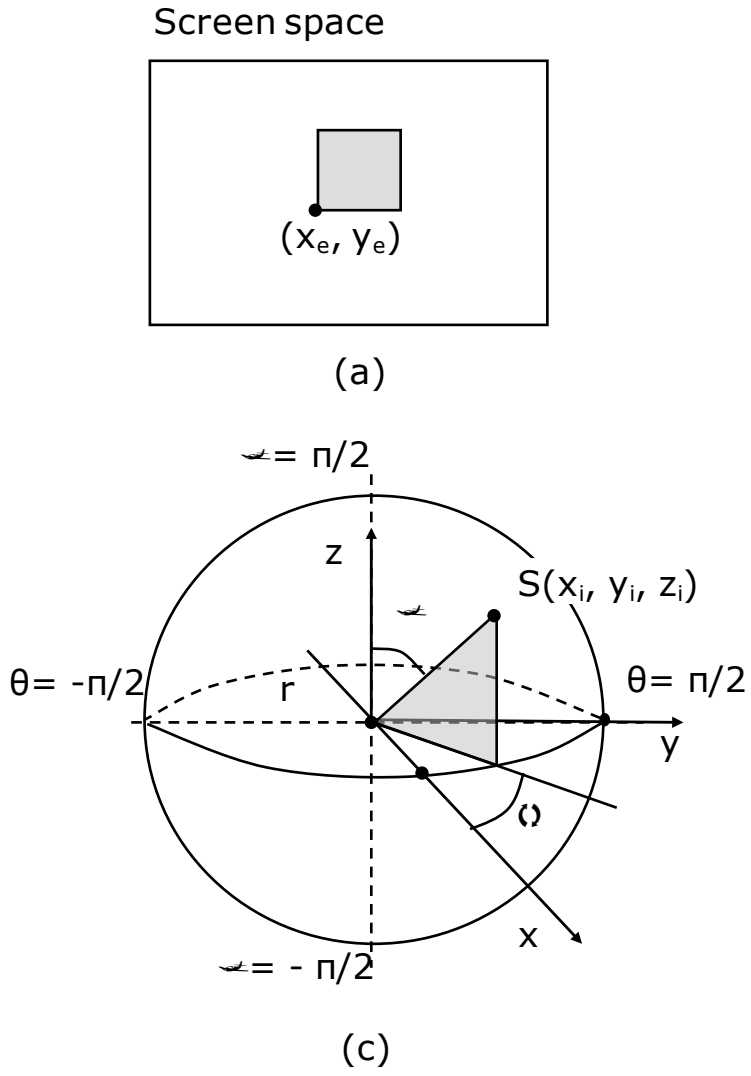


Two phase inverse mapping – example

- Intermediate surface: sphere
- O-mapping: intermediate surface normal
- Inverse mapping:
  1. Inverse mapping for all 4 pixel vertices onto 4 object points (x_w, y_w, z_w), a→b
  2. O-mapping to compute the point (θ, φ) onto the sphere surface, b→c
  3. S-mapping to determine the correspondent position (u, v) in the texture domain for the (θ, φ) point, c→d



Two phase inverse mapping – example



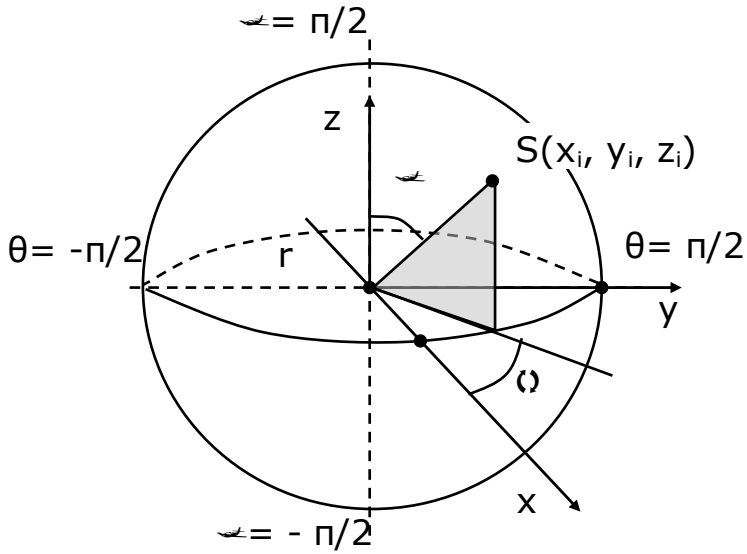
Two phase inverse mapping – example

$$\frac{x_w}{x_i} = \frac{y_w}{y_i} = \frac{z_w}{z_i} = \frac{\sqrt{x_w^2 + y_w^2 + z_w^2}}{r}$$

$$x_i = \frac{r\ x_w}{\sqrt{x_w^2 + y_w^2 + z_w^2}}$$

$$y_i = \frac{r\ y_w}{\sqrt{x_w^2 + y_w^2 + z_w^2}}$$

$$z_i = \frac{r\ z_w}{\sqrt{x_w^2 + y_w^2 + z_w^2}}$$



(c)

Compute the spherical coordinates θ and φ:

$tg\ \theta = y_i / x_i = y_w / x_w$, and then $\theta = arctg\ (y_w / x_w)$

$tg\ \varphi = [(x_w^2 + y_w^2)^{1/2} / z_w]$, and then $\varphi = arctg\ [(x_w^2 + y_w^2)^{1/2} / z_w]$

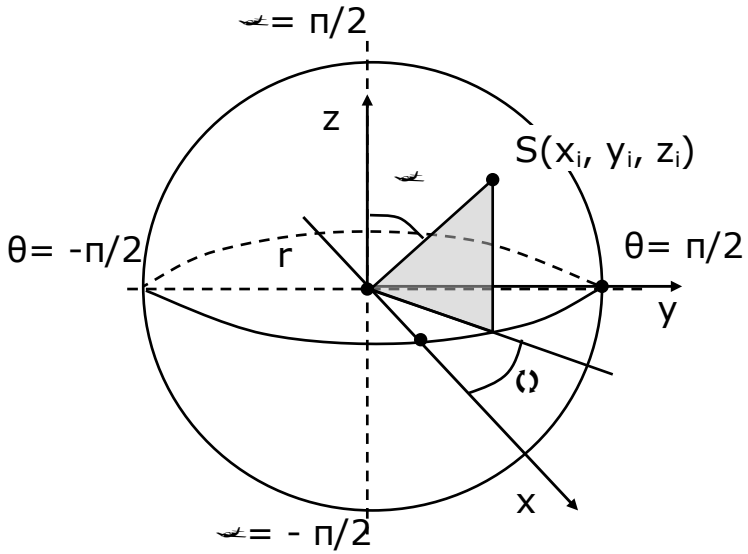
Two phase inverse mapping – example

Compute (u,v) by inverse S-mapping (θ,φ):

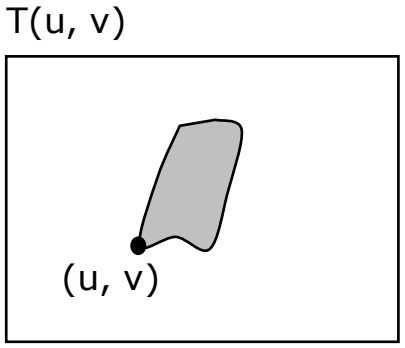
(θ,φ) ↜ (u, v)

$u = \theta / 2\pi = 1/2\pi \ \text{arctg} \ (y_w / x_w)$

$v = (\varphi + \pi/2) / \pi = 1/\pi \ \text{arctg} \ [(x_w^2 + y_w^2)^{1/2} / z_w] + 1/2$



(c)



(d)

Mapping through modeling

- Appropriate for objects that can be shaped from a deformed cylinder, sphere or another geometry
- Processing steps:
  1. Model the object as a set of cylinders and spheres
  2. Map the textures onto the object components – cylinders and spheres. The texture becomes fixed onto the object surface. The graphics model consists of bounded textures and objects
  3. Deformed by matrix based operations the objects and therefore the bounded textures

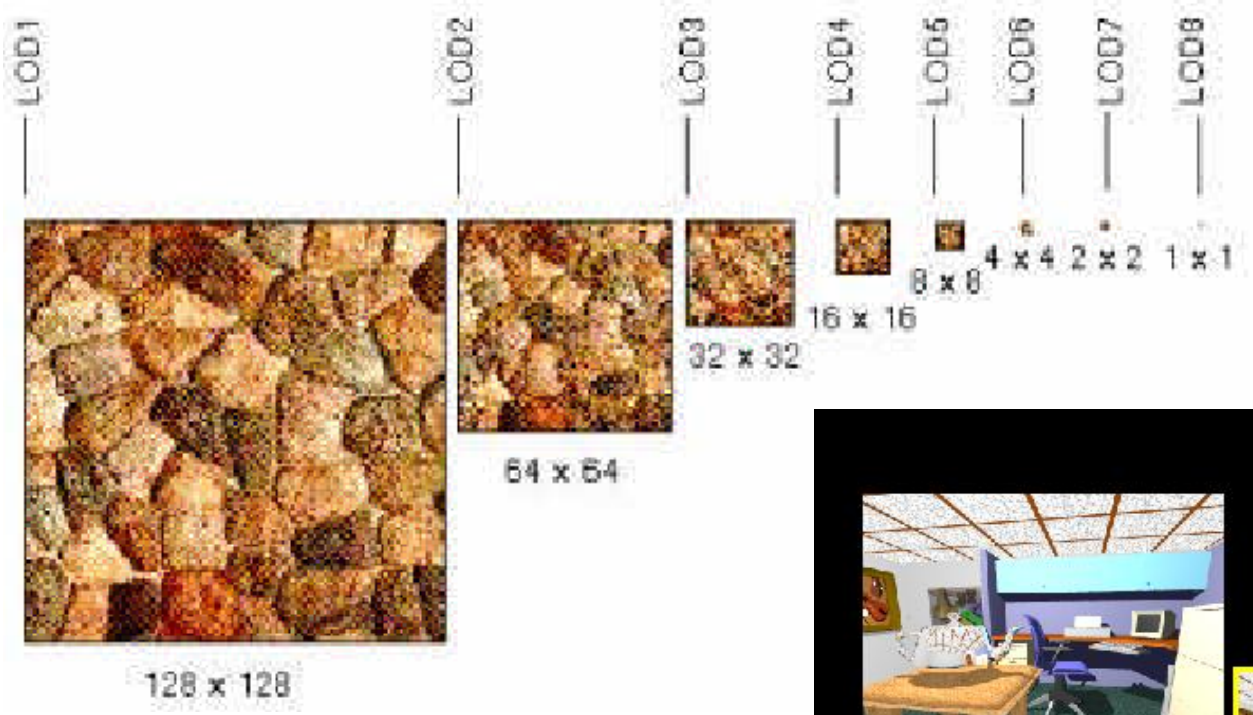
(a) (b) (c)

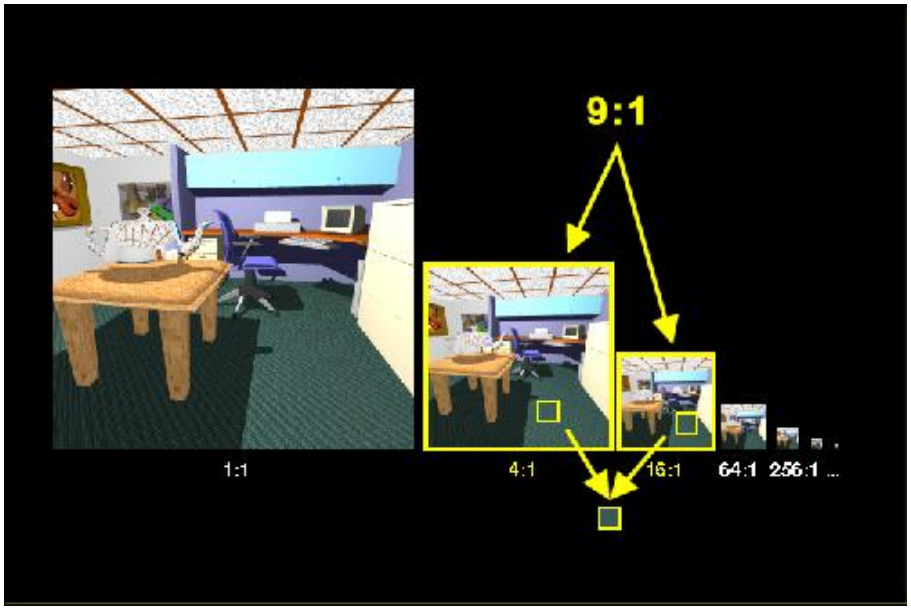
Mapping onto bicubic parametric patches

- Developed by Catmull in 1974
- Object is a bicubic parametric patch
- Texture mapping is trivial since a parametric patch definition is by u and v
- Mapping approach:
  1. Subdivide a surface patch in object space
  2. At the same time executes a corresponding subdivision in texture space
  3. Patch subdivision proceeds until it covers a single pixel
  4. The required texture value is obtained from the area enclosed by the current level of subdivision in the texture domain
- Another approach developed by Cook, Carpenter and Catmull in 1987
  - The object surfaces are subdivided into micro polygons and flat shaded with values from a corresponding subdivision in texture space

Mipmapping

- Texture depends on the LOD (Level of Details)





Questions and problems

1. Explain and exemplify the terms “texturing” and “texture mapping”.
2. What means the terms “fix texture” and “view dependent mapping”?
3. Explain the terms “surface parameterization” and “projection” of the direct texture mapping process.
4. Explain the antialiasing process. Why the rasterisation process loose information?
5. Explain the reason for the process of inverse mapping.
6. Explain the texture mapping onto a surface if the function of correspondence is: (a) linear toward two directions; (b) non-linear.
7. Compute the inverse mapping of a spherical surface for

$$0 < \theta \leq \pi/2 \text{ and } \pi/4 < \varphi \leq \pi/2$$

Questions and problems

8. Propose a rule based O mapping in order to resolve the main issues for the two phase inverse mapping onto concave 3D objects.
9. Why the two phase inverse mapping considers a pixel as a polygon?
10. Explain how the texture mapping could depends on LOD (Level Of Details)?